

Introducción a JavaScript asíncrono

[Descripción general: JavaScript asíncrono](#)[Próximo →](#)

En este artículo, explicaremos qué es la programación asíncrona, por qué la necesitamos y analizaremos brevemente algunas de las formas en que históricamente se han implementado las funciones asíncronas en JavaScript.

Requisitos previos:	Un sólido conocimiento de los fundamentos de JavaScript .
Resultados del aprendizaje:	• Para familiarizarnos con qué es JavaScript asíncrono, en qué se diferencia de JavaScript síncrono y por qué lo necesitamos. • Qué es la programación síncrona y por qué a veces puede resultar problemática. • Cómo la programación asíncrona pretende resolver estos problemas. • Los gestores de eventos y las funciones de devolución de llamada, y su relación con la programación asíncrona.

La programación asíncrona es una técnica que permite que tu programa inicie una tarea potencialmente larga y siga respondiendo a otros eventos mientras se ejecuta, en lugar de tener que esperar a que finalice. Una vez finalizada la tarea, tu programa recibe el resultado.

Muchas funciones que ofrecen los navegadores, especialmente las más interesantes, pueden tardar mucho tiempo en ejecutarse y, por lo tanto, son asíncronas. Por ejemplo:

- Realizar solicitudes HTTP usando `fetch()`
- Acceder a la cámara o al micrófono de un usuario mediante `getUserMedia()`
- Solicitar a un usuario que seleccione archivos usando `showOpenFilePicker()`

Por lo tanto, aunque no tengas que *implementar* tus propias funciones asíncronas muy a menudo, es muy probable que necesites usarlas *correctamente* .

En este artículo, comenzaremos analizando el problema de las funciones síncronas de larga duración, que hacen que la programación asíncrona sea una necesidad.

Programación síncrona

Considere el siguiente código:

JS

```
const name = "Miriam";
const greeting = `Hello, my name is ${name}!`;
```

```
console.log(greeting);  
// "Hello, my name is Miriam!"
```

Este código:

1. Declara una cadena llamada `name`.
2. Declara otra cadena llamada `greeting`, que utiliza `name`.
3. Muestra el saludo en la consola de JavaScript.

Cabe destacar que el navegador ejecuta el programa línea por línea, en el orden en que lo escribimos. En cada paso, espera a que la línea termine su trabajo antes de continuar con la siguiente. Esto es necesario porque cada línea depende del trabajo realizado en las líneas anteriores.

Eso convierte a este **programa en síncrono**. Seguiría siendo síncrono incluso si llamáramos a una función separada, como esta:

JS

```
function makeGreeting(name) {  
  return `Hello, my name is ${name}!`;  
}  
  
const name = "Miriam";  
const greeting = makeGreeting(name);  
console.log(greeting);  
// "Hello, my name is Miriam!"
```

En este caso, `makeGreeting()` se trata de una **función síncrona** porque quien la llama tiene que esperar a que la función termine su trabajo y devuelva un valor antes de poder continuar.

Una función síncrona de larga duración

¿Qué ocurre si la función síncrona tarda mucho tiempo?

El programa que se muestra a continuación utiliza un algoritmo muy ineficiente para generar varios números primos grandes cuando el usuario hace clic en el botón "Generar primos". Cuanto mayor sea la cantidad de primos que especifique el usuario, más tiempo tardará la operación.

HTML

```
<label for="quota">Number of primes:</label>  
<input type="text" id="quota" name="quota" value="1000000" />  
  
<button id="generate">Generate primes</button>  
<button id="reload">Reload</button>  
  
<div id="output"></div>
```

JS

```
const MAX_PRIME = 1000000;

function isPrime(n) {
  for (let i = 2; i <= Math.sqrt(n); i++) {
    if (n % i === 0) {
      return false;
    }
  }
  return n > 1;
}

const random = (max) => Math.floor(Math.random() * max);

function generatePrimes(quota) {
  const primes = [];
  while (primes.length < quota) {
    const candidate = random(MAX_PRIME);
    if (isPrime(candidate)) {
      primes.push(candidate);
    }
  }
  return primes;
}

const quota = document.querySelector("#quota");
const output = document.querySelector("#output");

document.querySelector("#generate").addEventListener("click", () => {
  const primes = generatePrimes(quota.value);
  output.textContent = `Finished generating ${quota.value} primes!`;
});

document.querySelector("#reload").addEventListener("click", () => {
  document.location.reload();
});
```

Number of primes: Generate primesReload

Prueba a hacer clic en "Generar números primos". Dependiendo de la velocidad de tu ordenador, probablemente tardará unos segundos en aparecer el mensaje "¡Finalizado!".

El problema de las funciones síncronas de larga duración

El siguiente ejemplo es igual que el anterior, solo que hemos añadido un cuadro de texto para que escribas. Esta vez, haz clic en "Generar números primos" e intenta escribir en el cuadro de texto inmediatamente después.

Comprobarás que, **mientras nuestra `generatePrimes()` función se está ejecutando, nuestro programa no responde en absoluto**: no puedes escribir nada, hacer clic en nada ni realizar ninguna otra acción.

Number of primes:

Generate primes

Reload

Try typing in here immediately after pressing "Generate primes"

Esto se debe a que este programa JavaScript es *de un solo hilo*. Un hilo es una secuencia de instrucciones que sigue un programa. Dado que el programa consta de un solo hilo, solo puede realizar una tarea a la vez; por lo tanto, si está esperando a que finalice nuestra llamada síncrona de larga duración, no puede hacer nada más.

Lo que necesitamos es una manera para que nuestro programa:

1. **Inicie una operación de larga duración llamando a una función.**
2. **Haz que esa función inicie la operación y regrese inmediatamente, para que nuestro programa pueda seguir respondiendo a otros eventos.**
3. **La función debe ejecutar la operación de forma que no bloquee el hilo principal, por ejemplo, iniciando un nuevo hilo.**
4. **Infórmenos del resultado de la operación una vez finalizada.**

Eso es precisamente lo que nos permiten hacer las funciones asíncronas. El resto de este módulo explica cómo se implementan en JavaScript.

Controladores de eventos

La descripción que acabamos de ver de las funciones asíncronas podría recordarte a los manejadores de eventos, y si es así, estás en lo cierto. Los manejadores de eventos son, en realidad, una forma de programación asíncrona: se proporciona una función (el manejador de eventos) que se llamará, no de inmediato, sino cuando ocurra el evento. Si "el evento" es "la operación asíncrona ha finalizado", entonces ese evento podría usarse para notificar a quien realiza la llamada sobre el resultado de la misma.

Algunas de las primeras API asíncronas utilizaban eventos precisamente de esta manera. La `XMLHttpRequest` API permite realizar solicitudes HTTP a un servidor remoto mediante JavaScript. Dado que esto puede tardar bastante, se

trata de una API asíncrona, y se recibe notificación sobre el progreso y la finalización de la solicitud mediante la adición de detectores de eventos al `XMLHttpRequest` objeto.

El siguiente ejemplo muestra esto en acción. Pulse "Haga clic para iniciar la solicitud" para enviar una solicitud. Creamos una nueva `XMLHttpRequest` y escuchamos su `loadend` evento. El controlador registra un mensaje "¡Finalizado!" junto con el código de estado.

Tras añadir el detector de eventos, enviamos la solicitud. Cabe destacar que, después de esto, podemos registrar "Solicitud XHR iniciada": es decir, nuestro programa puede seguir ejecutándose mientras se procesa la solicitud, y nuestro controlador de eventos se llamará cuando esta finalice.

HTML

```
<button id="xhr">Click to start request</button>
<button id="reload">Reload</button>

<pre class="event-log"></pre>
```

JS

```
const log = document.querySelector(".event-log");

document.querySelector("#xhr").addEventListener("click", () => {
  log.textContent = "";

  const xhr = new XMLHttpRequest();

  xhr.addEventListener("loadend", () => {
    log.textContent = `${log.textContent}Finished with status: ${xhr.status}`;
  });

  xhr.open(
    "GET",
    "https://raw.githubusercontent.com/mdn/content/main/files/en-us/_wikihistory.json",
  );
  xhr.send();
  log.textContent = `${log.textContent}Started XHR request\n`;
});

document.querySelector("#reload").addEventListener("click", () => {
  log.textContent = "";
  document.location.reload();
});
```

Click to start requestReload

Se trata de un [controlador de eventos](#), al igual que los controladores de acciones del usuario, como cuando el usuario hace clic en un botón. Sin embargo, en este caso, el evento es un cambio en el estado de un objeto.

Devoluciones de llamada

Un controlador de eventos es un tipo particular de función de devolución de llamada. [Una función de devolución de llamada es simplemente una función que se pasa a otra función, con la expectativa de que se ejecute en el momento adecuado.](#) Como acabamos de ver, las funciones de devolución de llamada solían ser la principal forma de implementar funciones asíncronas en JavaScript.

Sin embargo, el código basado en funciones de devolución de llamada puede resultar difícil de entender cuando la propia función de devolución de llamada debe llamar a funciones que aceptan otra función de devolución de llamada. Esta es una situación común si se necesita realizar alguna operación que se divide en una serie de funciones asíncronas. Por ejemplo, considere lo siguiente:

JS

```
function doStep1(init) {  
  return init + 1;  
}  
  
function doStep2(init) {  
  return init + 2;  
}  
  
function doStep3(init) {  
  return init + 3;  
}  
  
function doOperation() {  
  let result = 0;  
  result = doStep1(result);  
  result = doStep2(result);  
  result = doStep3(result);  
  console.log(`result: ${result}`);  
}  
  
doOperation();
```

Aquí tenemos una única operación dividida en tres pasos, donde cada paso depende del anterior. En nuestro ejemplo, el primer paso suma 1 a la entrada, el segundo suma 2 y el tercero suma 3. Partiendo de una entrada de 0, el resultado final es 6 ($0 + 1 + 2 + 3$). Como programa síncrono, esto es muy sencillo. Pero, ¿qué pasaría si implementáramos los pasos usando funciones de devolución de llamada?

JS

```
function doStep1(init, callback) {
  const result = init + 1;
  callback(result);
}

function doStep2(init, callback) {
  const result = init + 2;
  callback(result);
}

function doStep3(init, callback) {
  const result = init + 3;
  callback(result);
}

function doOperation() {
  doStep1(0, (result1) => {
    doStep2(result1, (result2) => {
      doStep3(result2, (result3) => {
        console.log(`result: ${result3}`);
      });
    });
  });
}

doOperation();
```

Debido a que tenemos que llamar a funciones de devolución de llamada dentro de otras funciones de devolución de llamada, obtenemos una `doOperation()` función anidada muy compleja, lo que dificulta enormemente su lectura y depuración. A esto se le suele llamar "el infierno de las funciones de devolución de llamada" o "la pirámide de la perdición" (porque la indentación se asemeja a una pirámide de lado).

Cuando anidamos funciones de devolución de llamada de esta manera, también puede resultar muy difícil manejar los errores: a menudo hay que manejar los errores en cada nivel de la "pirámide", en lugar de tener el manejo de errores solo una vez en el nivel superior.

Por estas razones, la mayoría de las API asíncronas modernas no utilizan devoluciones de llamada. En cambio, la base de la programación asíncrona en JavaScript es el `Promise`, y ese es el tema del próximo artículo.

[Descripción general: JavaScript asíncrono](#)[Próximo →](#)



Tu plan para una mejor internet.